In [ ]:

```
# ============================================================
# DEADHUNT — FULL ANALYSIS & BETTING OPTIMIZER
# Copy-paste into your notebook and run cell by cell.
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import norm, lognorm, shapiro, kstest, anderson
from scipy.optimize import minimize
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams['figure.figsize'] = (14, 6)

df = pd.read_csv('race_data.csv')   # <-- adjust path if needed
horses = ['Shadowfax','Iron Duke','Morningstar','Red Tide',
          'Gallant Fox','Blue Streak','Copper Prince','Last Chance']

market_probs = {
    'Shadowfax': 0.08, 'Iron Duke': 0.09, 'Morningstar': 0.11,
    'Red Tide': 0.13,  'Gallant Fox': 0.14, 'Blue Streak': 0.15,
    'Copper Prince': 0.14, 'Last Chance': 0.16
}
TAKEOUT   = 0.15
BANKROLL  = 10_000

# ── verify winner column ──────────────────────────────────
assert (df[horses].idxmin(axis=1) == df['winner']).all(), "winner mismatch!"
print("✓ Winner column verified\n")

# ═══════════════════════════════════════════════════════════
# 1. BASIC STATISTICS
# ═══════════════════════════════════════════════════════════
print("="*60)
print("1. DESCRIPTIVE STATISTICS")
print("="*60)
stats_df = df[horses].describe().T
stats_df['skew']     = df[horses].skew()
stats_df['kurtosis'] = df[horses].kurtosis()
stats_df['cv']       = stats_df['std'] / stats_df['mean']   # coeff of variati
stats_df['iqr']      = df[horses].quantile(0.75) - df[horses].quantile(0.25)
print(stats_df[['mean','std','cv','min','25%','50%','75%','max','skew','kurtos
      .sort_values('mean').round(4))

# ═══════════════════════════════════════════════════════════
# 2. EMPIRICAL WIN RATES  (with 95% CI via binomial)
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("2. EMPIRICAL WIN RATES vs MARKET")
```

```
print("="*60)
n = len(df)
win_counts = df['winner'].value_counts()
emp = {}
for h in horses:
    p_hat = win_counts.get(h, 0) / n
    se    = np.sqrt(p_hat * (1 - p_hat) / n)
    emp[h] = {'emp_prob': p_hat, 'ci_lo': max(0, p_hat - 1.96*se),
              'ci_hi': min(1, p_hat + 1.96*se), 'market': market_probs[h]}

emp_df = pd.DataFrame(emp).T.sort_values('emp_prob', ascending=False)
emp_df['edge_raw'] = emp_df['emp_prob'] - emp_df['market']
emp_df['ev']       = emp_df['emp_prob'] * (1 - TAKEOUT) / emp_df['market'] - 1
print(emp_df.round(4))

fig, ax = plt.subplots()
x = np.arange(len(horses))
ax.bar(x - 0.2, emp_df['emp_prob'], 0.35, label='Empirical', color='steelblue'
ax.bar(x + 0.2, emp_df['market'],   0.35, label='Market',    color='coral')
ax.errorbar(x - 0.2, emp_df['emp_prob'],
            yerr=[emp_df['emp_prob']-emp_df['ci_lo'], emp_df['ci_hi']-emp_df['
            fmt='none', color='black', capsize=4)
ax.set_xticks(x); ax.set_xticklabels(emp_df.index, rotation=30, ha='right')
ax.set_title("Empirical Win Rate vs Market (with 95% CI)"); ax.legend()
plt.tight_layout(); plt.show()

# ═══════════════════════════════════════════════════════════════
# 3. DISTRIBUTION FITTING — Normal, Log-Normal, Gamma
# ═══════════════════════════════════════════════════════════════
print("\n" + "="*60)
print("3. DISTRIBUTION FITTING")
print("="*60)

from scipy.stats import gamma as gamma_dist

fit_results = {}
for h in horses:
    t = df[h].values
    # fit all three
    mu, sigma     = t.mean(), t.std()
    norm_ll       = norm.logpdf(t, mu, sigma).sum()

    sh_ln, loc_ln, sc_ln = lognorm.fit(t, floc=0)
    lognorm_ll    = lognorm.logpdf(t, sh_ln, loc_ln, sc_ln).sum()

    sh_g, loc_g, sc_g = gamma_dist.fit(t, floc=0)
    gamma_ll      = gamma_dist.logpdf(t, sh_g, loc_g, sc_g).sum()

    best = max([('Normal', norm_ll), ('LogNormal', lognorm_ll), ('Gamma', gamm
               key=lambda x: x[1])

    # Shapiro-Wilk
    _, sw_p = shapiro(t)
```

```
    # KS vs Normal
    _, ks_p = kstest(t, 'norm', args=(mu, sigma))

    fit_results[h] = {
        'best_fit': best[0],
        'norm_ll': round(norm_ll,2), 'lognorm_ll': round(lognorm_ll,2), 'gamma
        'shapiro_p': round(sw_p,4), 'ks_p': round(ks_p,4),
        'norm_params': (mu, sigma),
        'ln_params':   (sh_ln, loc_ln, sc_ln),
        'gam_params':  (sh_g, loc_g, sc_g),
    }
    print(f"{h:<15} → {best[0]:<10}  (Normal LL={norm_ll:.1f}, LN LL={lognorm_
          f"Gamma LL={gamma_ll:.1f})  Shapiro p={sw_p:.4f}")

# ═══════════════════════════════════════════════════════════
# 4. QQ-PLOTS FOR EACH HORSE
# ═══════════════════════════════════════════════════════════
fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, h in zip(axes.flatten(), horses):
    stats.probplot(df[h], dist='norm', plot=ax)
    ax.set_title(f"{h} — QQ (Normal)")
plt.suptitle("QQ-Plots (Normal)", y=1.01); plt.tight_layout(); plt.show()

# ═══════════════════════════════════════════════════════════
# 5. KDE + FITTED DISTRIBUTION OVERLAY
# ═══════════════════════════════════════════════════════════
fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, h in zip(axes.flatten(), horses):
    t = df[h].values
    ax.hist(t, bins=30, density=True, alpha=0.4, color='steelblue')
    xs = np.linspace(t.min(), t.max(), 300)
    # Normal
    mu, sigma = fit_results[h]['norm_params']
    ax.plot(xs, norm.pdf(xs, mu, sigma), 'r-', lw=1.5, label='Normal')
    # LogNormal
    sh, loc, sc = fit_results[h]['ln_params']
    ax.plot(xs, lognorm.pdf(xs, sh, loc, sc), 'g--', lw=1.5, label='LogNormal'
    ax.set_title(h); ax.legend(fontsize=7)
plt.suptitle("Histogram + Fitted Distributions", y=1.01); plt.tight_layout();

# ═══════════════════════════════════════════════════════════
# 6. CORRELATION ANALYSIS
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("6. CORRELATION BETWEEN HORSE TIMES")
print("="*60)
corr = df[horses].corr()
print(corr.round(3))
# Note: near-zero correlations → horses are independent → simulate independent

fig, ax = plt.subplots(figsize=(10,8))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', ax=ax,
            vmin=-1, vmax=1, center=0)
```

```
ax.set_title("Correlation Between Finishing Times"); plt.tight_layout(); plt.s

# ═══════════════════════════════════════════════════════════
# 7. LARGE MONTE CARLO SIMULATION (best-fit per horse)
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("7. MONTE CARLO SIMULATION (500,000 races)")
print("="*60)
N_SIMS = 500_000
np.random.seed(42)
sim = np.zeros((N_SIMS, len(horses)))
for i, h in enumerate(horses):
    fr = fit_results[h]
    if fr['best_fit'] == 'Normal':
        mu, sig = fr['norm_params']
        sim[:, i] = np.random.normal(mu, sig, N_SIMS)
    elif fr['best_fit'] == 'LogNormal':
        sh, loc, sc = fr['ln_params']
        sim[:, i] = lognorm.rvs(sh, loc, sc, size=N_SIMS)
    else:
        sh, loc, sc = fr['gam_params']
        sim[:, i] = gamma_dist.rvs(sh, loc, sc, size=N_SIMS)

win_idx = np.argmin(sim, axis=1)
mc_probs = {horses[i]: (win_idx == i).mean() for i in range(len(horses))}
print(f"{'Horse':<15} {'MC Prob':>9} {'Market':>9} {'EV':>9}")
for h in horses:
    p  = mc_probs[h]
    m  = market_probs[h]
    ev = p * (1 - TAKEOUT) / m - 1
    print(f"{h:<15} {p:>9.4f} {m:>9.4f} {ev:>+9.4f}")

# ═══════════════════════════════════════════════════════════
# 8. BOOTSTRAP CONFIDENCE INTERVALS ON MC PROBS
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("8. BOOTSTRAP CI ON MC WIN PROBABILITIES")
print("="*60)
N_BOOT = 500
boot_probs = {h: [] for h in horses}
sample_size = 500
for _ in range(N_BOOT):
    sample = df[horses].sample(n=sample_size, replace=True).values
    w_idx  = np.argmin(sample, axis=1)
    for i, h in enumerate(horses):
        boot_probs[h].append((w_idx == i).mean())

print(f"{'Horse':<15} {'Mean':>8} {'CI_Lo':>8} {'CI_Hi':>8}")
boot_ci = {}
for h in horses:
    arr  = np.array(boot_probs[h])
    lo, hi = np.percentile(arr, [2.5, 97.5])
    boot_ci[h] = (arr.mean(), lo, hi)
```

```
    print(f"{h:<15} {arr.mean():>8.4f} {lo:>8.4f} {hi:>8.4f}")

# ═══════════════════════════════════════════════════════════════
# 9. HEAD-TO-HEAD WIN MATRIX
# ═══════════════════════════════════════════════════════════════
print("\n" + "="*60)
print("9. HEAD-TO-HEAD DOMINANCE MATRIX (% horse A beats horse B)")
print("="*60)
h2h = pd.DataFrame(index=horses, columns=horses, dtype=float)
for i, ha in enumerate(horses):
    for j, hb in enumerate(horses):
        h2h.loc[ha, hb] = (df[ha] < df[hb]).mean()
print(h2h.round(3))

fig, ax = plt.subplots(figsize=(10,8))
sns.heatmap(h2h.astype(float), annot=True, fmt='.2f', cmap='RdYlGn', ax=ax, vm
ax.set_title("Head-to-Head: P(row beats col)"); plt.tight_layout(); plt.show()

# ═══════════════════════════════════════════════════════════════
# 10. PLACE / SHOW PROBABILITIES (top-2, top-3 finishes)
# ═══════════════════════════════════════════════════════════════
print("\n" + "="*60)
print("10. TOP-2 AND TOP-3 FINISH PROBABILITIES (from simulation)")
print("="*60)
rank_matrix = np.argsort(sim, axis=1)           # shape (N_SIMS, 8), rank[i,j]
# rank_matrix[:,i] = ranks of horse i across all sims
ranks = np.zeros_like(sim, dtype=int)
for i in range(N_SIMS):
    ranks[i, rank_matrix[i]] = np.arange(len(horses))

print(f"{'Horse':<15} {'Win':>7} {'Top-2':>7} {'Top-3':>7}")
for i, h in enumerate(horses):
    w  = (ranks[:, i] == 0).mean()
    t2 = (ranks[:, i] <= 1).mean()
    t3 = (ranks[:, i] <= 2).mean()
    print(f"{h:<15} {w:>7.3f} {t2:>7.3f} {t3:>7.3f}")

# ═══════════════════════════════════════════════════════════════
# 11. STREAK ANALYSIS — recent form (last 50 / last 100 races)
# ═══════════════════════════════════════════════════════════════
print("\n" + "="*60)
print("11. RECENT FORM (last 50 vs last 100 races)")
print("="*60)
for window, label in [(50,'Last 50'), (100,'Last 100'), (500,'All 500')]:
    subset = df.tail(window)
    wc = subset['winner'].value_counts()
    print(f"\n  {label}:")
    for h in horses:
        print(f"    {h:<15}: {wc.get(h,0):>3} wins  ({wc.get(h,0)/window:.1%})

# Rolling win rate chart
fig, ax = plt.subplots(figsize=(14,5))
for h in horses:
```

```
    roll = (df['winner'] == h).rolling(50).mean()
    ax.plot(roll, label=h)
ax.axhline(1/8, color='black', ls='--', lw=1, label='Uniform 12.5%')
ax.set_title("50-Race Rolling Win Rate"); ax.legend(ncol=4, fontsize=8)
plt.tight_layout(); plt.show()

# ═══════════════════════════════════════════════════════════════
# 12. PACE / FINISHING TIME TRENDS OVER RACES
# ═══════════════════════════════════════════════════════════════
print("\n" + "="*60)
print("12. PACE TREND — is any horse getting faster/slower?")
print("="*60)
for h in horses:
    slope, intercept, r, p, se = stats.linregress(df['race_id'], df[h])
    print(f"{h:<15} slope={slope:+.5f} s/race   p={p:.4f}  {'*SIGNIFICANT*' if

fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, h in zip(axes.flatten(), horses):
    ax.scatter(df['race_id'], df[h], s=2, alpha=0.4)
    m, b, *_ = stats.linregress(df['race_id'], df[h])
    ax.plot(df['race_id'], m*df['race_id']+b, 'r-', lw=1.5)
    ax.set_title(h); ax.set_xlabel('Race'); ax.set_ylabel('Time (s)')
plt.suptitle("Finishing Time Trend per Horse", y=1.01); plt.tight_layout(); pl

# ═══════════════════════════════════════════════════════════════
# 13. VOLATILITY ANALYSIS — variance of rank positions
# ═══════════════════════════════════════════════════════════════
print("\n" + "="*60)
print("13. RANK VOLATILITY (std dev of finishing rank across 500 races)")
print("="*60)
rank_df = df[horses].rank(axis=1)   # 1 = fastest
print(f"{'Horse':<15} {'Mean Rank':>10} {'Std Rank':>10}  (lower mean & std =
for h in horses:
    print(f"{h:<15} {rank_df[h].mean():>10.3f} {rank_df[h].std():>10.3f}")

# ═══════════════════════════════════════════════════════════════
# 14. KELLY CRITERION — OPTIMAL BET SIZING
# ═══════════════════════════════════════════════════════════════
print("\n" + "="*60)
print("14. KELLY CRITERION ANALYSIS")
print("="*60)

# Use MC probs
evs = {h: mc_probs[h] * (1-TAKEOUT) / market_probs[h] - 1 for h in horses}
pos_ev = [h for h in horses if evs[h] > 0]

print(f"\nPositive EV horses: {pos_ev}")
print(f"\n{'Horse':<15} {'p_true':>8} {'f_mkt':>8} {'EV':>8}")
for h in horses:
    print(f"{h:<15} {mc_probs[h]:>8.4f} {market_probs[h]:>8.4f} {evs[h]:>+8.4f

# Simultaneous Kelly for parimutuel (Smoczynski & Tomkins formula)
sum_p     = sum(mc_probs[h] for h in pos_ev)
```

```python
sum_f_adj = sum(market_probs[h] / (1-TAKEOUT) for h in pos_ev)
Q         = (1 - sum_p) / (1 - sum_f_adj)

kelly_fracs = {}
for h in pos_ev:
    f = mc_probs[h] - (market_probs[h] / (1-TAKEOUT)) * Q
    kelly_fracs[h] = max(0, f)

print(f"\nSimultaneous Kelly fractions:")
for h in horses:
    kf = kelly_fracs.get(h, 0)
    print(f"  {h:<15}: {kf:.4f}  →  £{kf*BANKROLL:,.0f}")

# ═══════════════════════════════════════════════════════════
# 15. STRATEGY COMPARISON & SIMULATION
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("15. STRATEGY COMPARISON (simulated P&L)")
print("="*60)

def sim_strategy(stakes_dict, n=100_000):
    """Simulate average profit per race."""
    stakes = np.array([stakes_dict.get(h, 0) for h in horses])
    payouts = stakes * (1 - TAKEOUT) / np.array([market_probs[h] for h in hors
    profits_per_outcome = payouts - stakes.sum()   # net if that horse wins
    # add back the stake on the winning horse
    profits_per_outcome = payouts - stakes.sum()

    # simulate
    np.random.seed(0)
    sim2 = np.zeros((n, len(horses)))
    for i, h in enumerate(horses):
        fr = fit_results[h]
        if fr['best_fit'] == 'Normal':
            mu, sig = fr['norm_params']
            sim2[:, i] = np.random.normal(mu, sig, n)
        elif fr['best_fit'] == 'LogNormal':
            sh, loc, sc = fr['ln_params']
            sim2[:, i] = lognorm.rvs(sh, loc, sc, size=n)
        else:
            sh, loc, sc = fr['gam_params']
            sim2[:, i] = gamma_dist.rvs(sh, loc, sc, size=n)

    winners2 = np.argmin(sim2, axis=1)
    total_stake = stakes.sum()
    profits = np.array([payouts[w] - total_stake for w in winners2])
    return profits.mean(), profits.std(), (profits > 0).mean()

strategies = {}

# S1: All-in on best EV horse
best_ev_horse = max(evs, key=evs.get)
strategies['AllIn_BestEV'] = {best_ev_horse: BANKROLL}
```

```python
# S2: Pure Kelly
total_kf = sum(kelly_fracs.values())
strategies['Full_Kelly'] = {h: kelly_fracs.get(h,0) * BANKROLL for h in horses

# S3: Half Kelly (safer)
strategies['Half_Kelly'] = {h: kelly_fracs.get(h,0) * BANKROLL * 0.5 for h in

# S4: Proportional to true win prob (pos EV only)
strategies['Prob_Weight'] = {h: (mc_probs[h]/sum_p)*BANKROLL if h in pos_ev el

# S5: 70% Norm Kelly + 30% Prob
norm_kelly = {h: (kelly_fracs.get(h,0)/total_kf)*BANKROLL if total_kf>0 else 0
strategies['Clubbed_70_30'] = {h: 0.7*norm_kelly[h] + 0.3*strategies['Prob_Wei

# S6: Equal split on pos EV
strategies['Equal_PosEV'] = {h: BANKROLL/len(pos_ev) if h in pos_ev else 0 for

print(f"\n{'Strategy':<20} {'Avg Profit':>12} {'Std Dev':>10} {'Win Rate':>10}
best_strat, best_avg = None, -np.inf
for name, stk in strategies.items():
    avg, std, wr = sim_strategy(stk)
    print(f"{name:<20} {avg:>+12.2f} {std:>10.2f} {wr:>10.2%}")
    if avg > best_avg:
        best_avg, best_strat = avg, name

print(f"\n★  Best strategy by avg profit: {best_strat}  (£{best_avg:+.2f}/race

# ═══════════════════════════════════════════════════════════════
# 16. FINAL RECOMMENDED STAKES
# ═══════════════════════════════════════════════════════════════
print("\n" + "="*60)
print("16. FINAL RECOMMENDED STAKES")
print("="*60)

final = strategies[best_strat]
# round and ensure sum <= BANKROLL
final_int = {h: int(v) for h, v in final.items()}
leftover  = BANKROLL - sum(final_int.values())
final_int[best_ev_horse] = final_int.get(best_ev_horse, 0) + leftover

print(f"\nBased on strategy: {best_strat}")
print(f"{'Horse':<15} {'Stake £':>10} {'Payout if Win £':>18} {'True P':>8} {'
for h in horses:
    stk = final_int.get(h, 0)
    payout = stk * (1-TAKEOUT) / market_probs[h] if stk > 0 else 0
    print(f"{h:<15} {stk:>10,} {payout:>18,.0f} {mc_probs[h]:>8.3f} {market_pr

print(f"\nTotal staked: £{sum(final_int.values()):,}")
print(f"Expected profit per race: £{best_avg:+.2f}")
print(f"Expected profit over 10k races: £{best_avg*10000:+,.0f}")

# ═══════════════════════════════════════════════════════════════
```

```python
# 17. SENSITIVITY ANALYSIS — how robust is the edge?
# ═══════════════════════════════════════════════════
print("\n" + "="*60)
print("17. SENSITIVITY: EV under different true-prob assumptions")
print("="*60)
# Vary the win prob ± 20% and see if edge flips
for h in pos_ev:
    for factor in [0.8, 0.9, 1.0, 1.1, 1.2]:
        adj_p = mc_probs[h] * factor
        adj_ev = adj_p * (1-TAKEOUT) / market_probs[h] - 1
        flag = "✓" if adj_ev > 0 else "✗"
        print(f"  {h:<15} p×{factor:.1f} → p={adj_p:.3f}  EV={adj_ev:+.4f}  {f
    print()
```